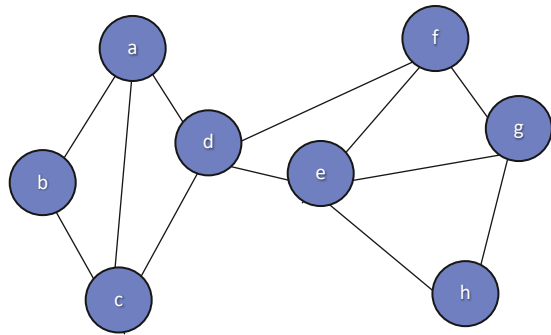


Графовые нейросети и их применение

Обозначения



$G = (V, E)$, где

G — структура, которая состоит из двух множеств

V — множество вершин и их признаков

E — множество ребер и их признаков

$|V|$ - количество вершин

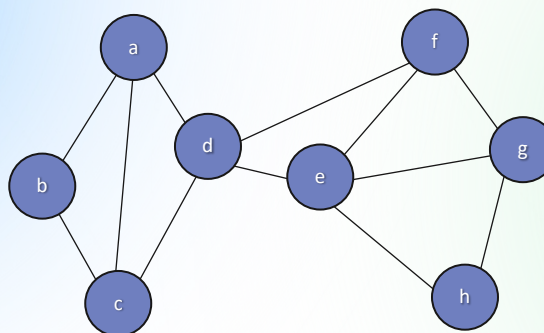
$|E|$ - количество ребер

u, v - вершины

d_u - степень вершины u

e_{uv} - признаки ребра, соединяющие вершины u, v

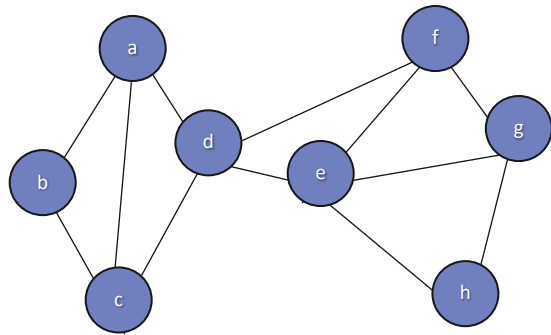
Динамические графы



+

t

Обозначения для динамических графов



$$G = (V(t), E(t), X(t)), \text{ где}$$

G — структура, которая состоит из двух множеств

V — множество вершин и их признаков

E — множество ребер и их признаков

X — множество событий

$|V|$ — количество вершин

$|E|$ — количество ребер

u, v — вершины

d_u — степень вершины u

e_{uv} — признаки ребра, соединяющие вершины u, v

t — непрерывное или дискретное время

Формализация типов динамики

Дискретная динамика (“snapshot”)

Граф задаётся последовательностью:

$$G_1, G_2, \dots, G_T$$

Каждый снимок — статический граф, но между ними структура меняется.

Проблемы:

- выбор длины окна
- потеря временной точности
- aliasing: разные события могут попадать в один снимок

Непрерывная динамика (“event-based”)

Граф — поток событий:

$$\mathcal{E} = \{(u, v, t, x_{uv}(t))\}$$

Парадигма “temporal point process”:

- каждое событие имеет точный timestamp
- интервалы между событиями несут информацию

Важные следствия:

- нужна память о прошлом
- нужна функция интенсивности событий
- порядок событий становится частью структуры

Какие задачи решают динамические GNN

1. Temporal Link Prediction

Предсказать, произойдёт ли событие между u и v в будущем:

$$\Pr[(u, v, t_{future}) | \mathcal{E}_{\leq t}]$$

2. Event Forecasting

Предсказать какое событие произойдёт следующим:

$$\Pr[(u, v, t + \Delta t) = e_k]$$

3. Node classification с изменяющимися атрибутами

Например, прогноз динамического статуса узлов:

$$y_v(t) \rightarrow y_v(t + \Delta t)$$

4. Комбинированные задачи

- динамическое community detection
- аномалии во времени
- прогноз временных рядов, “привязанных” к структуре графа

Challenges

1. Memory и долгосрочная зависимость

Требуется хранить состояние узла:

$$m_v(t) = f(m_v(t^-), \text{events involving } v)$$

Трудность: экспоненциальное затухание сигнала.

2. Эффективность

В event-based графах количество событий может быть $10^7 - 10^9$.

Нужны:

- локальные методы
- minibatch по событиям
- стохастическое обновление памяти

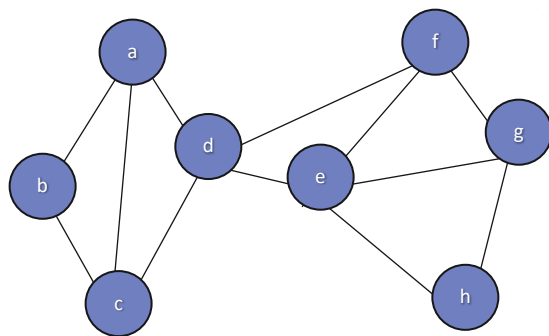
3. Нестационарность

Распределение событий меняется:

$$p_t(e) \neq p_{t-1}(e)$$

Статические модели быстро деградируют.

Комплексность архитектуры



$G = (V(t), E(t), X(t))$, где

G — структура, которая состоит из двух множеств

V — множество вершин и их признаков

E — множество ребер и их признаков

X — множество событий

t - непрерывное или дискретное время

Комбинация:

- GNN,
- RNN/GRU,
- self-attention,
- temporal encodings,
- memory modules.

Цель динамических GNN — построить отображение:

$$\Phi: \mathcal{G}_{\leq t} \rightarrow \mathbb{R}^d$$

такое, что представления узлов/рёбер/графа учитывают:

- порядок событий
- интервалы времени
- эволюцию структуры
- изменяющиеся признаки
- долгосрочную зависимость

Snapshot-based graphs

Формальное определение

Граф задан последовательностью статических снимков:

$$G_1, G_2, \dots, G_T, \quad G_t = (V_t, E_t)$$

Как строятся снeпшоты:

- фиксируем окно $[t; t + \Delta t]$;
- собираем все события внутри окна $\rightarrow$ получаем граф G_T

+ Плюсы

- Простая обработка (можно применять любые GNN).
- Подходит для задач node classification.
- Можно использовать RNN/attention вдоль времени.

– Минусы

- Потеря временной точности.
- Не учитывается порядок событий.
- Разные процессы могут давать одинаковый снeпшот (aliasing).
- Снeпшот — грубая аппроксимация.

Event-based / Continuous-time graphs

Формальное определение

Граф — поток событий:

$$\mathcal{E} = \{(u, v, t, x_{uv}(t)) \mid t \in \mathbb{R}_+\}$$

Каждое событие фиксирует:

- участников взаимодействия
- точную временную метку
- тип или параметры события

+ Плюсы

- Учитывается точное время
- Учитывается порядок событий
- Учитываются интервалы времени
- Естественно обрабатываются новые узлы

– Минусы

- Сложно обучать (много событий)
- Нужны модели с памятью
- Требуются минибатчи событий

Mixed / Semi-continuous

Варианты:

1. Snapshot + events inside snapshot

- структура обновляется по снелшотам
- но параметры/фичи узлов обновляются внутри окна

2. Discretized continuous-time

- время дискретизируется адаптивно
- внутри каждого интервала используется временной encoding

3. Edge streams + periodic GNN updates

- память обновляется по событиям
- GNN запускается редко

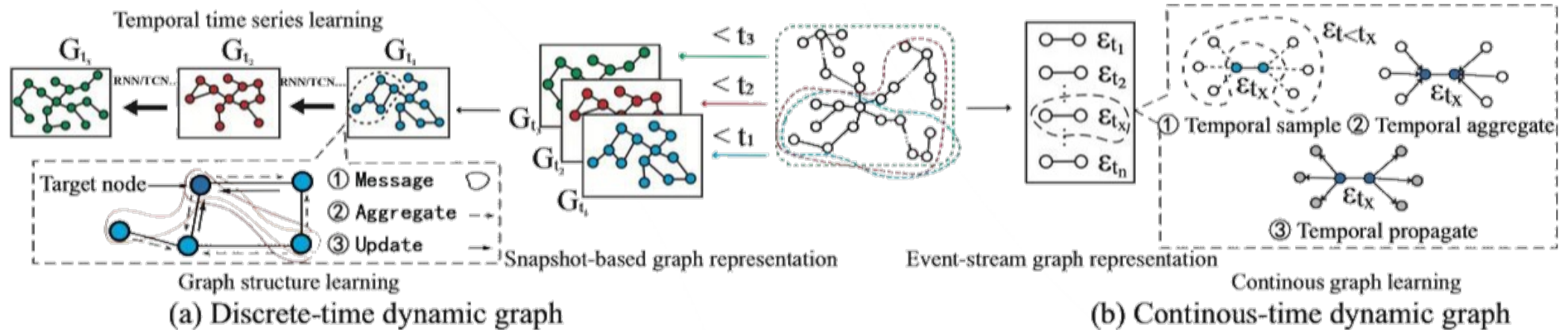
Когда полезно

- в real-time системах, где GNN слишком дорог
- при больших графах
- когда временная сетка есть физически (часовые слоты и т.п.)

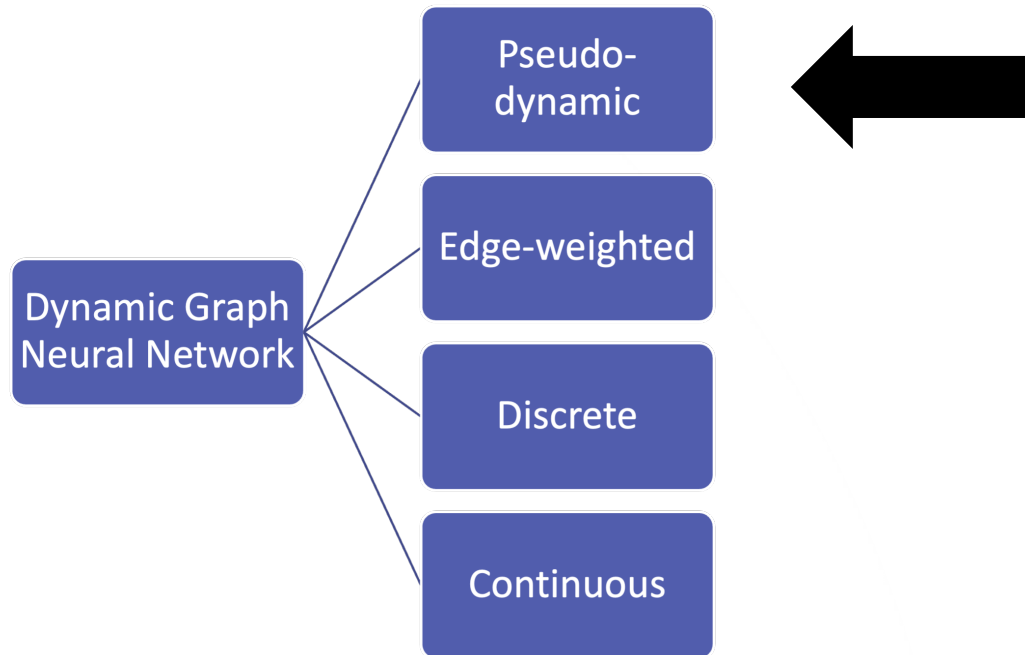
Сравнение типов динамики

Свойство	Snapshot	Event-based	Hybrid
Точность времени	низкая	максимальная	средняя
Порядок событий	нет	да	частично
Интервалы времени	нет	да	да
Масштабируемость	высокая	ниже	высокая
Новые узлы	трудно	легко	зависит
Реалистичность	средняя	максимальная	средняя/высокая

Типы DGNN



Типы DGNN



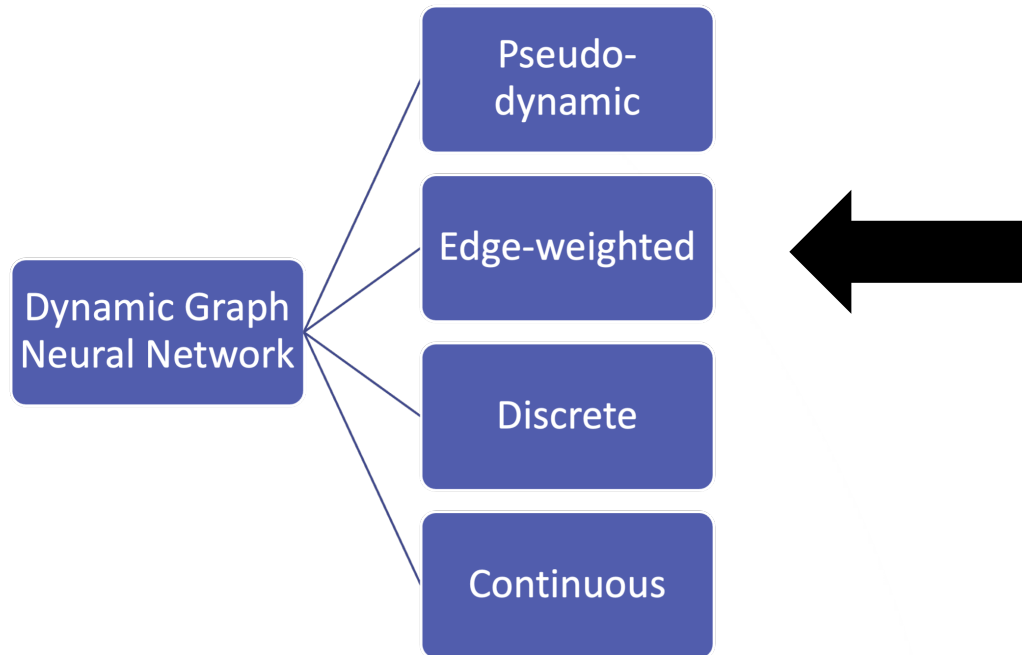
Пример:
Heterogeneous Graph Transformer

$$Base(\Delta T(t, s), 2i) = \sin\left(\Delta T_{t,s}/10000^{\frac{2i}{d}}\right) \quad (6)$$

$$Base(\Delta T(t, s), 2i + 1) = \cos\left(\Delta T_{t,s}/10000^{\frac{2i+1}{d}}\right) \quad (7)$$

$$RTE(\Delta T(t, s)) = \text{T-Linear}\left(Base(\Delta T_{t,s})\right) \quad (8)$$

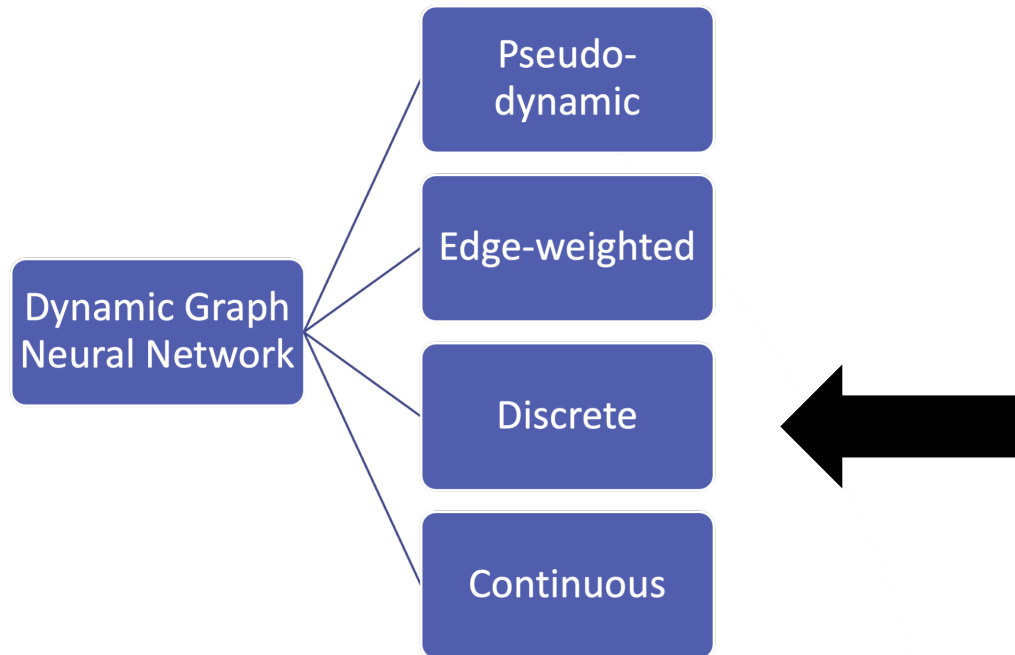
Типы DGNN



Пример:
Temporal Graph Transformer

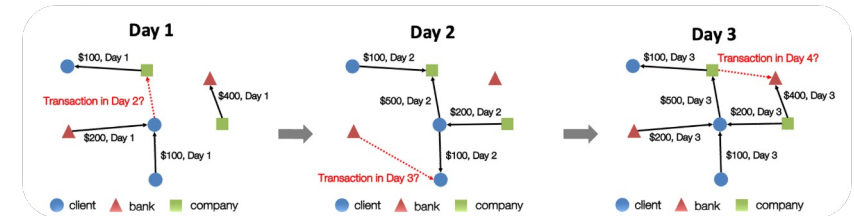
Итерационная нейросеть, где ребро, появившееся недавно получает более высокий вес, а ребро, появившееся давно – низкий вес

Типы DGNN

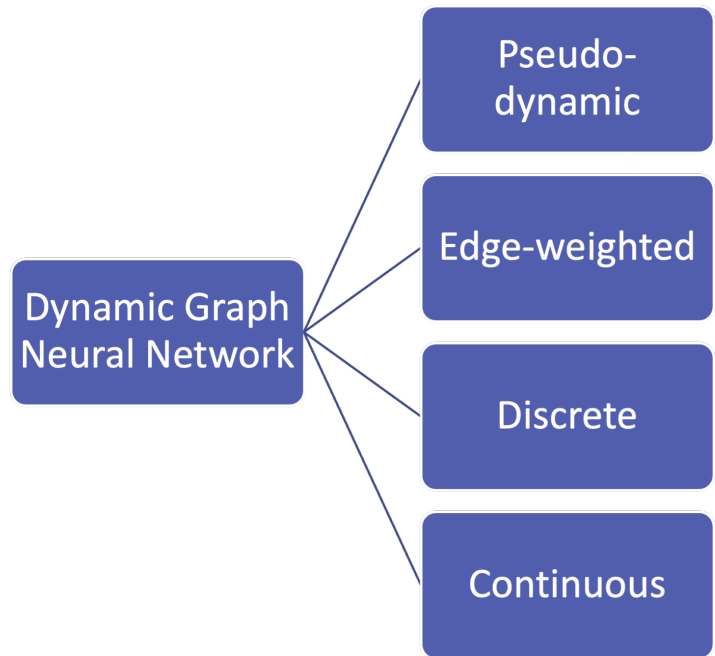


Динамический граф представлен в виде последовательности снимков, где каждый Снепшоп – статичный граф

$$G_t = (V_t, E_t):$$
$$G = (G_1, G_2, \dots, G_T)$$

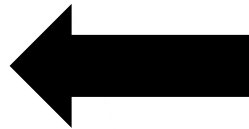


Типы DGNN

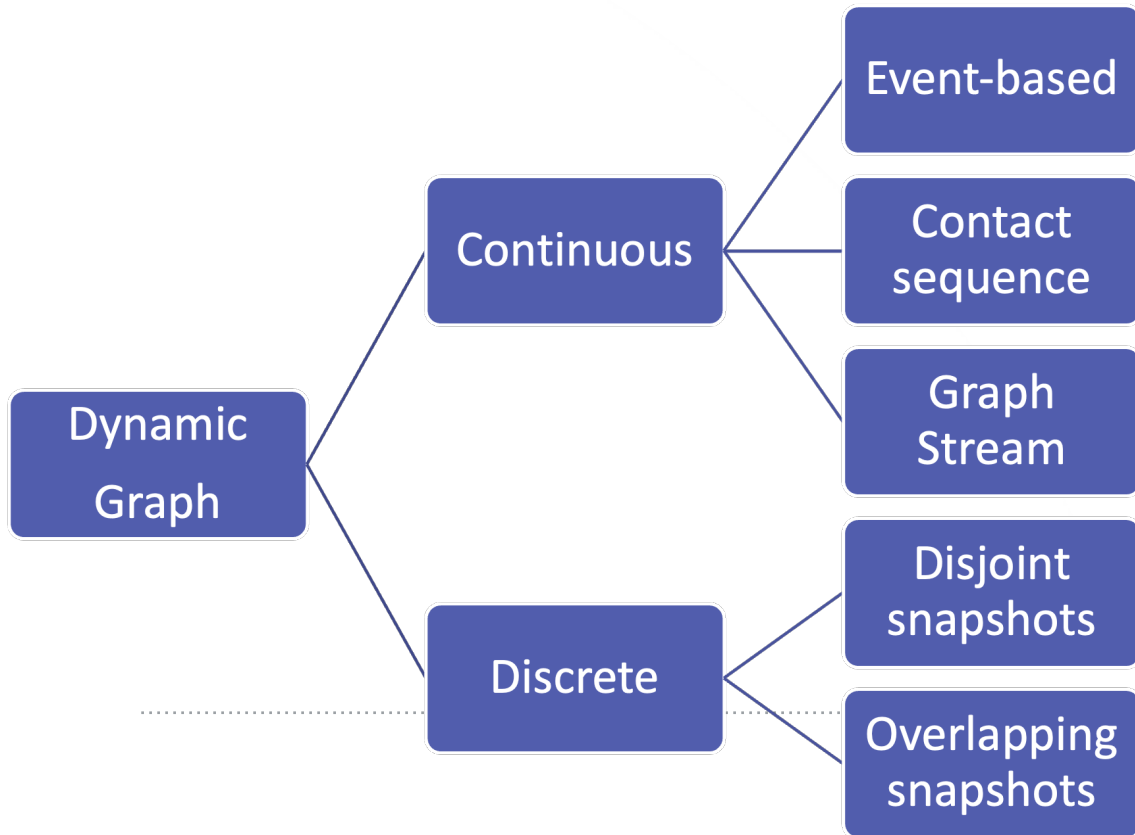


Различные модели с подходами непрерывного времени.

Например, в TGN динамический граф представлен в виде последовательности событий с временными метками.



Данные: непрерывные vs дискретные



Continuous Dynamic Graphs

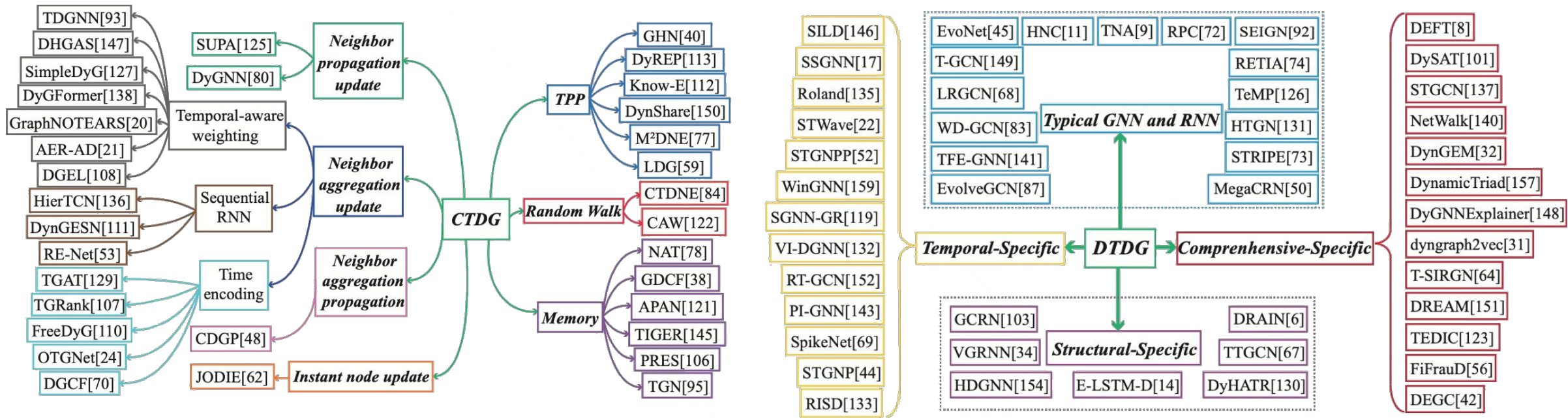
Continuous DGNN

Непрерывное представление может быть агрегировано в виде снимков

Discrete Dynamic Graphs

Discrete DGNN

Данные: непрерывные vs дискретные



Объединение времени и графов

Dynamic GNN

Time-Then-Graph (TTG)

$$\begin{aligned} \mathbf{H}_i^{\text{node}} &= \text{RNN}^{\text{node}}(\mathbf{X}_{i, \leq T}), \quad \forall i \in \mathcal{V}, \\ \mathbf{H}_{i,j}^{\text{edge}} &= \text{RNN}^{\text{edge}}(\mathbf{A}_{i,j, \leq T}), \quad \forall (i,j) \in \mathcal{E}, \\ \mathbf{Z} &= \text{GNN}^L(\mathbf{H}^{\text{node}}, \mathbf{H}^{\text{edge}}). \end{aligned}$$

Graph-Then-Time (GTT)

$$\mathbf{H}_{i,t} = \text{Cell} \left(\left[\text{GNN}_{\text{in}}^L(\mathbf{X}_{:,t}, \mathbf{A}_{:,:,t}) \right]_i, \mathbf{H}_{i,t-1} \right)$$

Time-And-Graph (TAG)

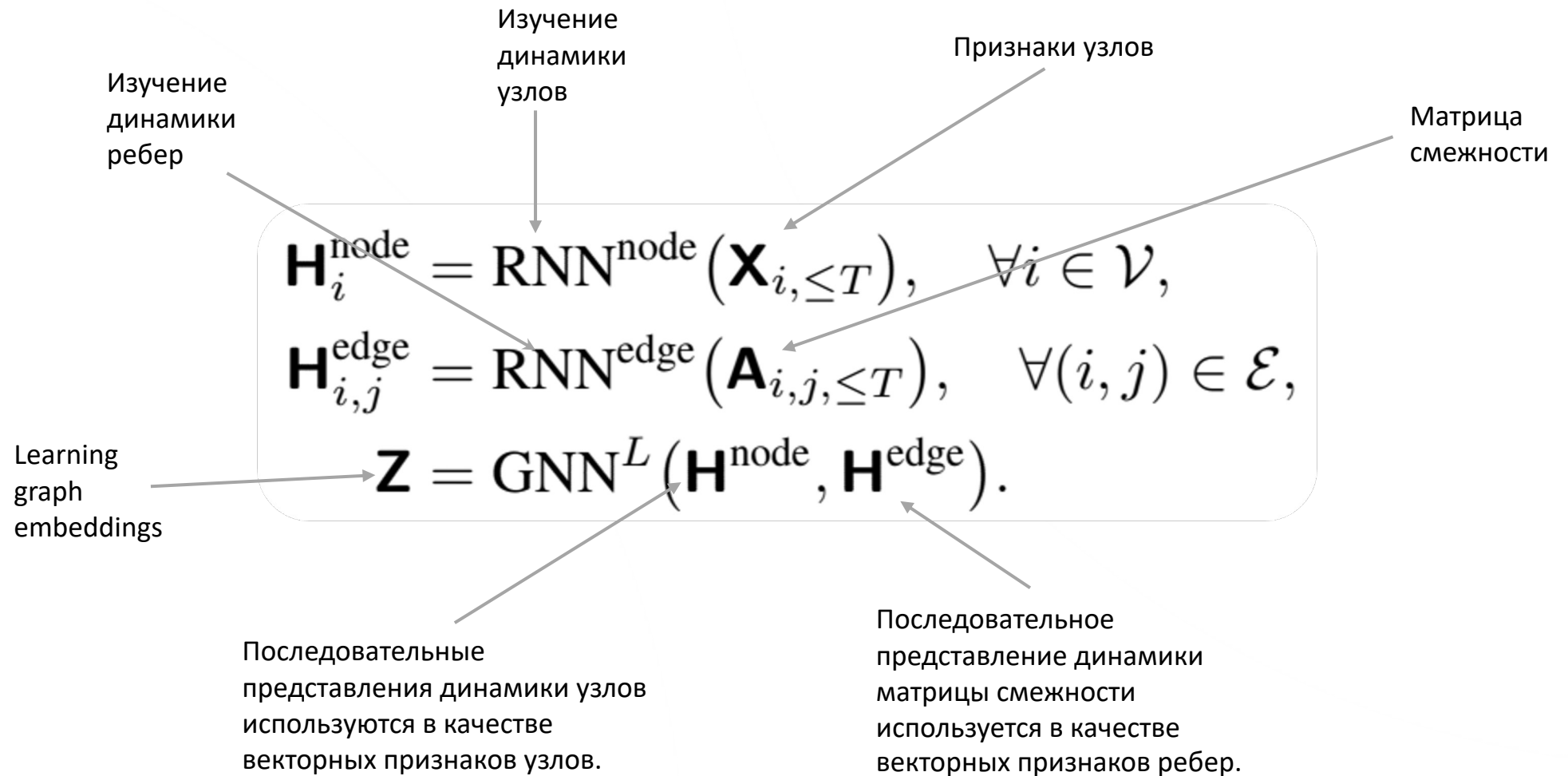
$$\mathbf{H}_{i,t} = \text{Cell} \left(\left[\text{GNN}_{\text{in}}^L(\mathbf{X}_{:,t}, \mathbf{A}_{:,:,t}) \right]_i, \left[\text{GNN}_{\text{rc}}^L(\mathbf{H}_{:,t-1}, \mathbf{A}_{:,:,t}) \right]_i \right),$$

Graph-And-Time DGNN

$H_{i,t}$ это временное и графовое представление временного узла i в момент времени t .

Cell (...) - is a RNN cell embedding evolutions of those GNN representation

Time-Then-Graph (TTG)



Time-And-Graph (TAG)



Graph-Then-Time (GTT)

GNN_{in} подготавливает графовые эмбединги для использования их в качестве входного представления для RNN

Признаки узлов

Матрица смежности

$$\mathbf{H}_{i,t} = \text{Cell} \left(\left[GNN_{in}^L(\mathbf{X}_{:,t}, \mathbf{A}_{:,:,t}) \right]_i, \mathbf{H}_{i,t-1} \right)$$

$H_{i,t}$ является временным и графовым представлением временного узла i в момент времени t .

Временное и графовое представление из предыдущей итерации RNN напрямую передается в качестве представления истории.

Graph-And-Time (GATG)



Карта соответствия: паттерны → архитектуры

Паттерн	Описание	Класс моделей
TTG	Сначала время, потом граф	редко используется, простые модели
GTT	GNN, потом RNN	GCRN, GCLSTM, DySAT (temporal block)
TAG	Spatio-temporal совместная обработка	ST-GCN, AGCRN, DCRNN
GATG	Непрерывное время + память + события	TGN, TGAT, DyRep, JODIE

Dynamic vs Static: архитектурные ограничения

Static GNNs

Имеет множество вариантов дизайна архитектуры, таких как:

- Эмбединги ребер
- skip-connections
- Различные нормализации
- Техники агрегации

...

Легко модифицируется и включает новые техники.

Dynamic GNNs

Извлечение информации из графов и выявление временных паттернов осуществляются отдельными процессами, что может привести к потере данных.

Сложно модифицировать и вводить новые техники в архитектуру.

Также сложно перенести успешные дизайны/архитектуры статических GNN на приложения для динамических графов.

Snapshot-based DGNN

GNN → RNN (GTT-паттерн)

Это основа snapshot-based DGNN.

На каждом шаге рассчитываем GNN:

$$Z_t = \text{GNN}(G_t)$$

Затем передаём состояние во времени:

$$H_t = \text{Cell}(Z_t, H_{t-1})$$

Эта схема определяет целый класс моделей.

GCRN / GCLSTM - GNN внутри рекуррентной ячейки

GCLSTM использует графовые операции **внутри LSTM**:

$$i_t = \sigma (W_i * A_t X_t + U_i * H_{t-1})$$

$$H_t = o_t \odot \tanh(C_t)$$

где * стоит для графовой фильтрации (diffusion или GCN-оператор).

Что это даёт?

- Узлы обновляют память с учётом своих соседей.
- Более "структурно-сознательная" память, чем обычный RNN.

Используется в:

- транспортных сетях,
- сенсорных сетях (DCRNN вариант),
- городских временных графах.

Spatio-Temporal GNN (TAG-паттерн): ST-GCN, DCRNN, AGCRN

Эти модели обрабатывают граф и время одновременно.

Пример ST-GCN (Yan et al.):

$$H^{(l,t)} = \sum_{\tau=-K}^K A_t \cdot H^{(l-1,t+\tau)} W_{\tau}$$

Интерпретация

- время — как дополнительное измерение,
- применяется 3D-свёртка по структуре и времени.

Сильные стороны

- отличны для периодических процессов (трафик, IoT),
- устойчивы к шумам.

Недостатки

- время обязательно дискретное,
- тяжёлые вычислительно.

EvolveGCN - эволюционирующие веса GNN (GTP-паттерн+)

Одна из ключевых snapshot-моделей.

Главная идея

Не только эмбединги узлов меняются, но **сами параметры GNN эволюционируют**:

$$W_t = \text{GRU}(W_{t-1}, \Phi(G_t))$$

Затем:

$$H_t = \sigma(A_t H_{t-1} W_t)$$

EvolveGCN - эволюционирующие веса GNN (GTP-паттерн+)

Два варианта архитектуры

EvolveGCN-H

- GNN-веса обновляются из узловых эмбедингов
- параметрический update

EvolveGCN-O

- предыдущие веса напрямую подаются в GNN
- output-driven update

Event-based DGNN

Event-based DGNN

Event-based DGNN — это модели **непрерывного времени**, которые обновляют состояние графа **в момент каждого события**, а не по снейшотам.

Они представляют граф как поток событий:

$$\mathcal{E} = \{(u, v, t, x_{uv}(t))\}$$

и обрабатывают события **точно в момент их наступления**.

Это — наиболее мощный и современный класс DGNN, соответствующий паттерну **GATG (Graph-And-Time)**.

Общая архитектура event-based DGNN

1) Memory module — память узлов

Хранит состояние каждого узла:

$$m_v(t)$$

Память обновляется **только в момент событий**, поэтому:

$$m_v(t) = m_v(t^-) \quad \text{если узел } v \text{ не участвует в событиях в момент } t$$

2) Message function — сообщения по событиям

Событие (u, v, t) создаёт сообщение:

$$\text{msg}(u, v, t) = \phi(m_u(t^-), m_v(t^-), x_{uv}(t), \Delta t)$$

Это сообщение используется для обновления памяти.

Общая архитектура event-based DGNN

3) Update function (RNN-like) — обновление памяти

$$m_v(t) = \text{Cell}(m_v(t^-), \text{msg}(u, v, t))$$

Это может быть GRU, LSTM или специализированный update (DyRep).

4) Embedding module — построение представлений узлов

После обновления памяти:

$$h_v(t) = \text{GNN}(m(t), A_{\text{local}}(t))$$

или через temporal-attention (TGAT).

Общая архитектура event-based DGNN

5) Time encoding

Важный компонент: как представить время?

В TGAT / TGN:

$$\phi(\Delta t) = [\sin(\omega_1 \Delta t), \cos(\omega_1 \Delta t), \dots]$$

В DyRep: экспоненты.

TGAT - Temporal Graph Attention Networks

1) Temporal attention

Зависимость между узлами определяется через attention с учетом времени:

$$\alpha_{u \rightarrow v}(t) = \text{softmax} (q_v^\top k_u(\Delta t))$$

2) Time encoding

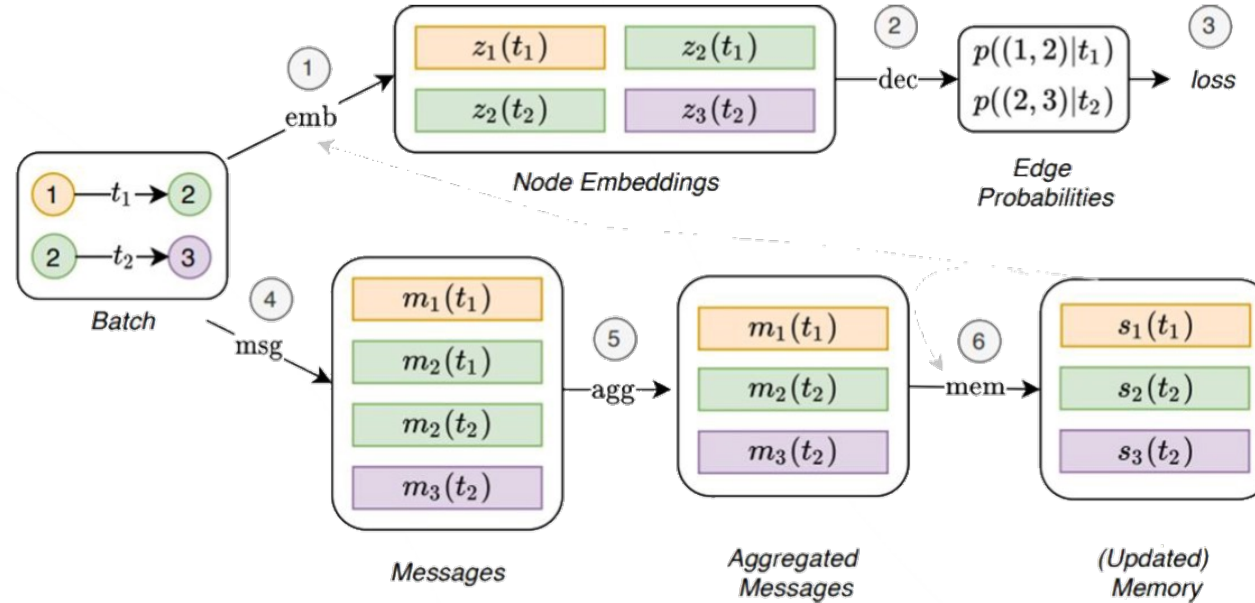
Использует синусоидальное кодирование времени (как positional encoding):

$$\phi(\Delta t) = [\sin(\omega \Delta t), \cos(\omega \Delta t)]$$

3) Neighborhood sampling

TGAT рассматривает временные эго-графы.

Architecture limitations: TGN



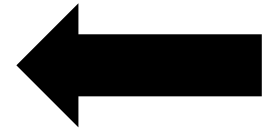
$$\mathbf{m}_i(t) = \text{msg}_s(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_d(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t))$$

$$\mathbf{m}_i(t) = \text{msg}_n(\mathbf{s}_i(t^-), t, \mathbf{v}_i(t)).$$

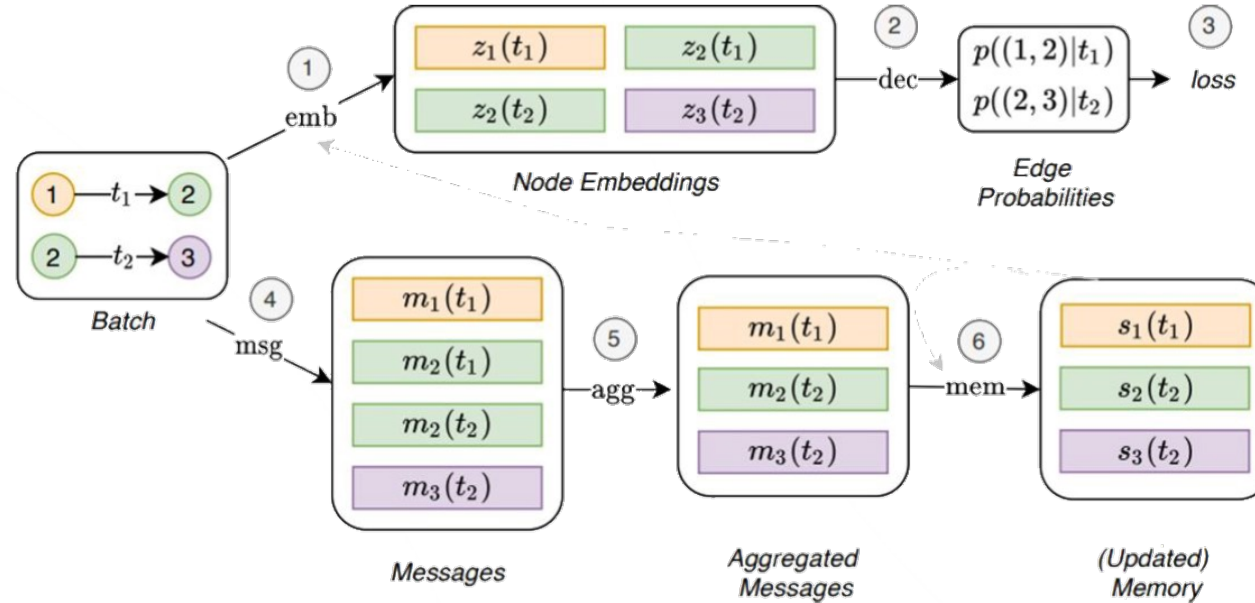
$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)).$$

$$\mathbf{s}_i(t) = \text{mem}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-)).$$

$$\mathbf{z}_i(t) = \text{emb}(i, t) = \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t)),$$



Architecture limitations: TGN



$$\mathbf{m}_i(t) = \text{msg}_s(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_d(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t))$$

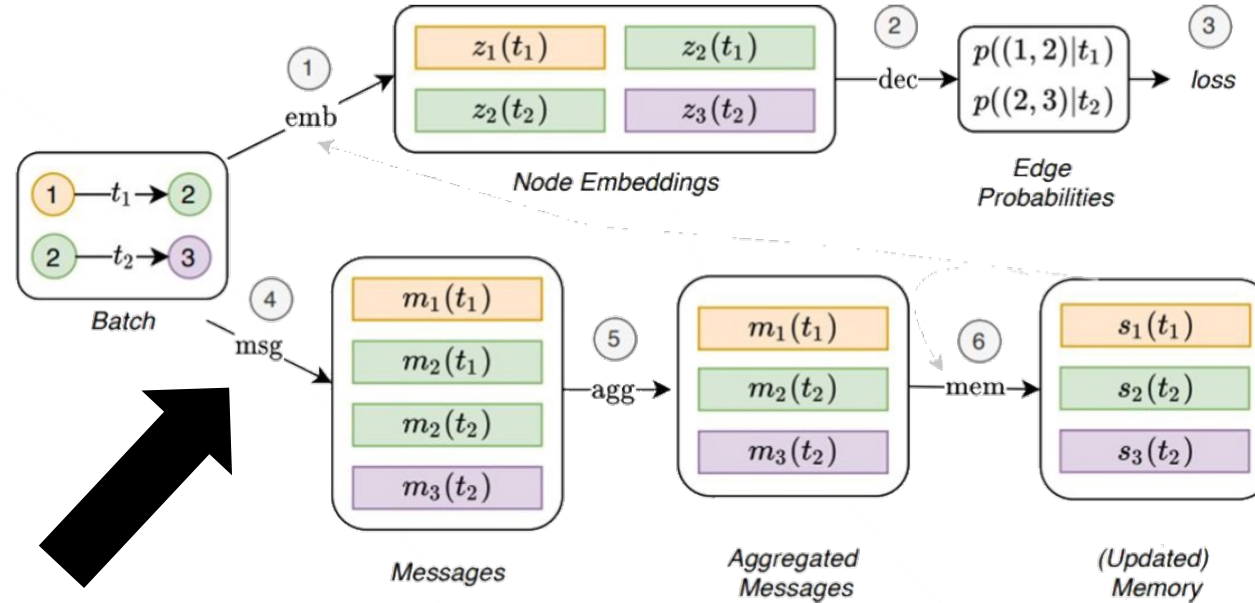
$$\mathbf{m}_i(t) = \text{msg}_n(\mathbf{s}_i(t^-), t, \mathbf{v}_i(t)).$$

$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)).$$

$$\mathbf{s}_i(t) = \text{mem}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-)).$$

$$\mathbf{z}_i(t) = \text{emb}(i, t) = \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t)),$$

Architecture limitations: TGN



$$\mathbf{m}_i(t) = \text{msg}_s(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_d(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t))$$

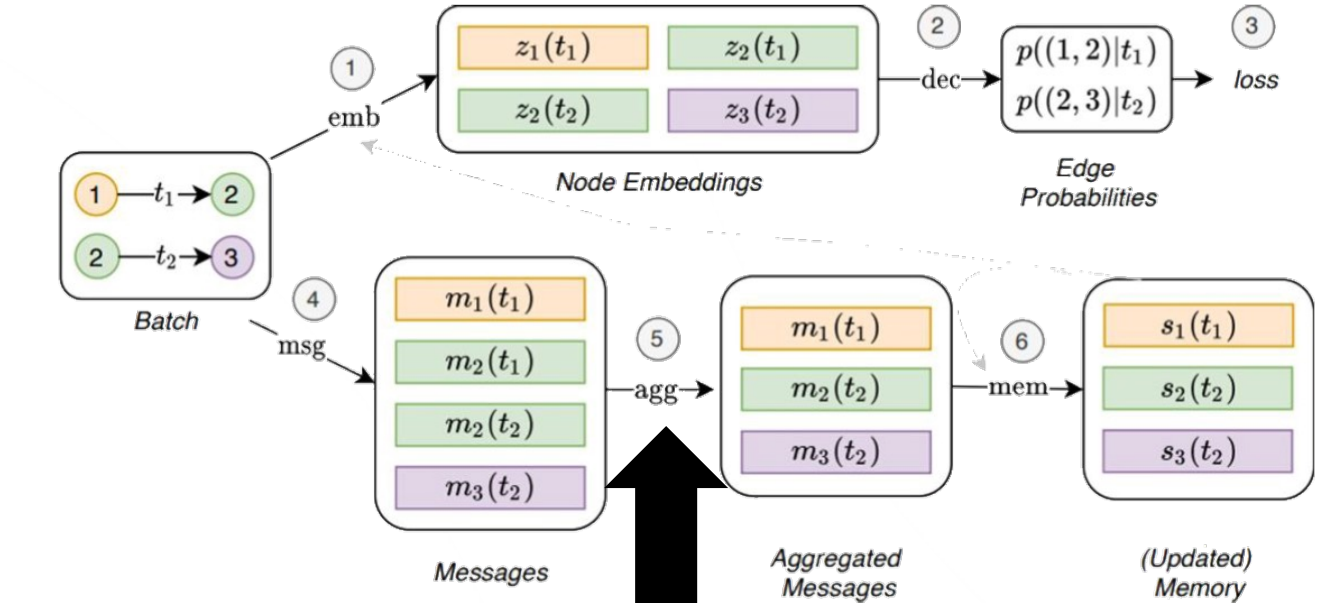
$$\mathbf{m}_i(t) = \text{msg}_n(\mathbf{s}_i(t^-), t, \mathbf{v}_i(t)).$$

$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)).$$

$$\mathbf{s}_i(t) = \text{mem}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-)).$$

$$\mathbf{z}_i(t) = \text{emb}(i, t) = \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t)),$$

Architecture limitations: TGN



$$\mathbf{m}_i(t) = \text{msg}_s(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_d(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t))$$

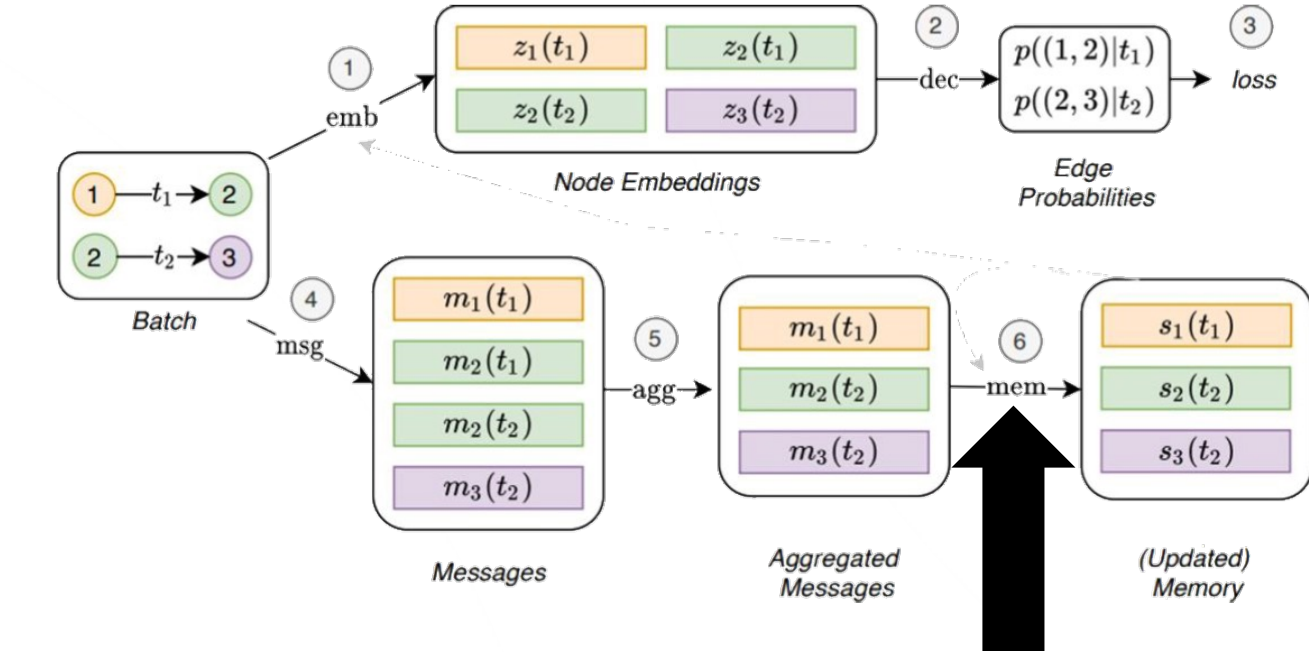
$$\mathbf{m}_i(t) = \text{msg}_n(\mathbf{s}_i(t^-), t, \mathbf{v}_i(t)).$$

$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)).$$

$$\mathbf{s}_i(t) = \text{mem}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-)).$$

$$\mathbf{z}_i(t) = \text{emb}(i, t) = \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t)),$$

Architecture limitations: TGN



$$\mathbf{m}_i(t) = \text{msg}_s(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_d(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t))$$

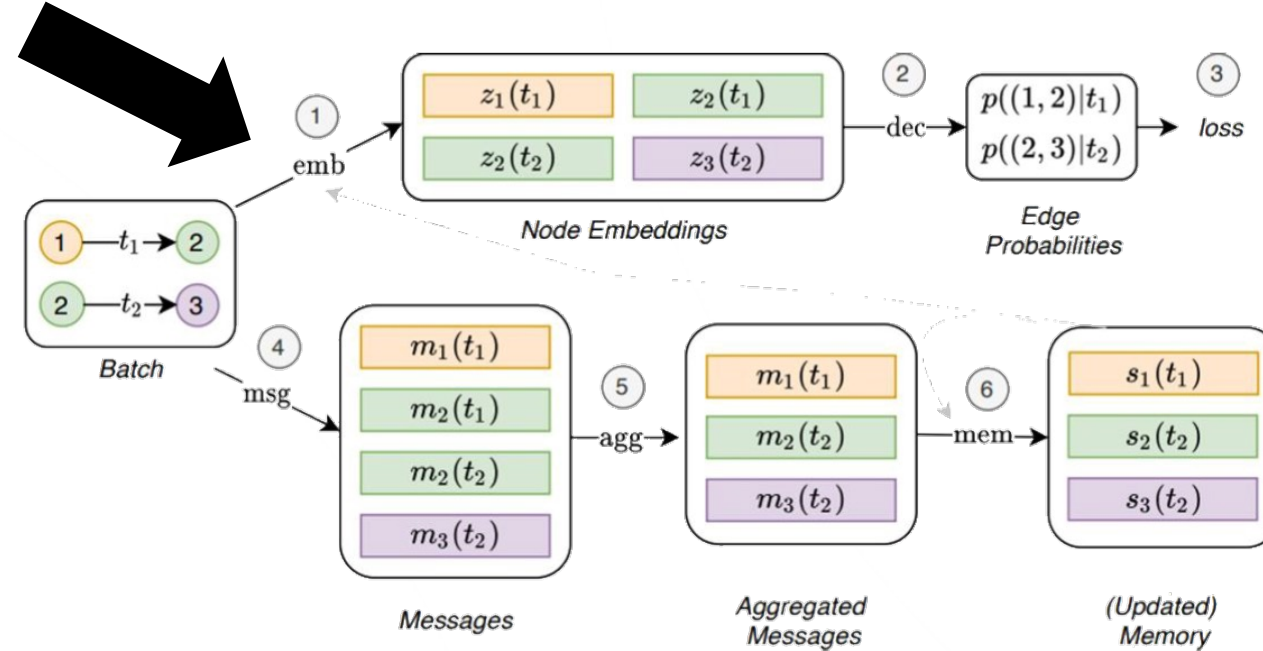
$$\mathbf{m}_i(t) = \text{msg}_n(\mathbf{s}_i(t^-), t, \mathbf{v}_i(t)).$$

$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)).$$

$$\mathbf{s}_i(t) = \text{mem}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-)).$$

$$\mathbf{z}_i(t) = \text{emb}(i, t) = \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t)),$$

Architecture limitations: TGN



$$\mathbf{m}_i(t) = \text{msg}_s(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_d(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t))$$

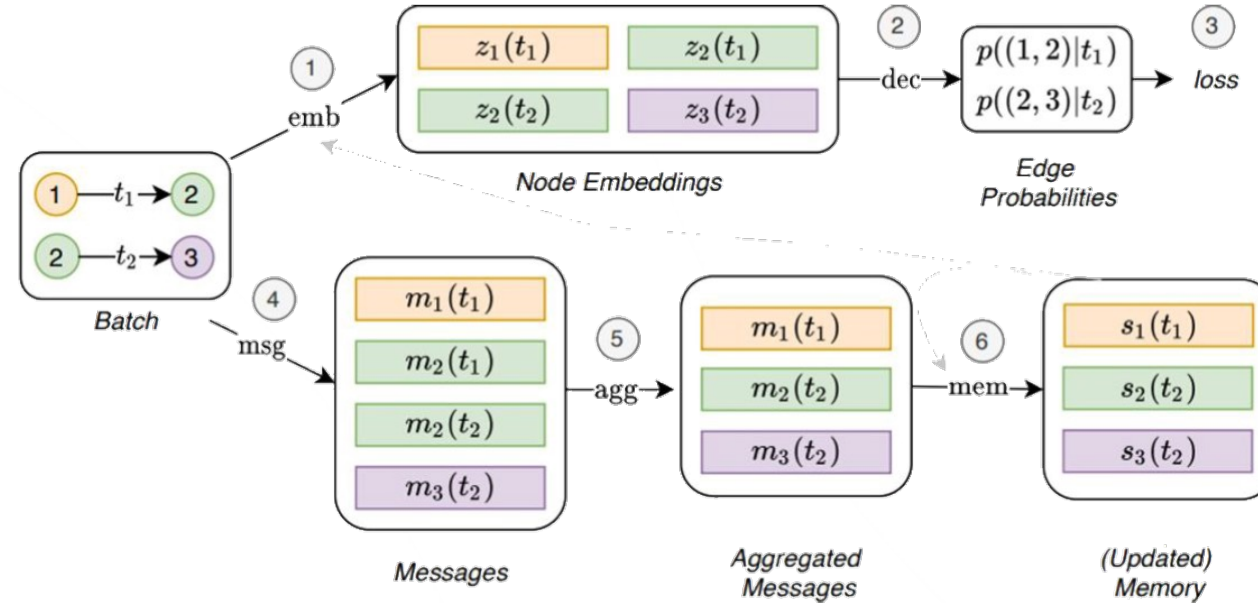
$$\mathbf{m}_i(t) = \text{msg}_n(\mathbf{s}_i(t^-), t, \mathbf{v}_i(t)).$$

$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)).$$

$$\mathbf{s}_i(t) = \text{mem}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-)).$$

$$\mathbf{z}_i(t) = \text{emb}(i, t) = \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t)),$$

Architecture limitations: TGN



$$\mathbf{m}_i(t) = \text{msg}_s(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_d(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t))$$

$$\mathbf{m}_i(t) = \text{msg}_n(\mathbf{s}_i(t^-), t, \mathbf{v}_i(t)).$$

$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)).$$

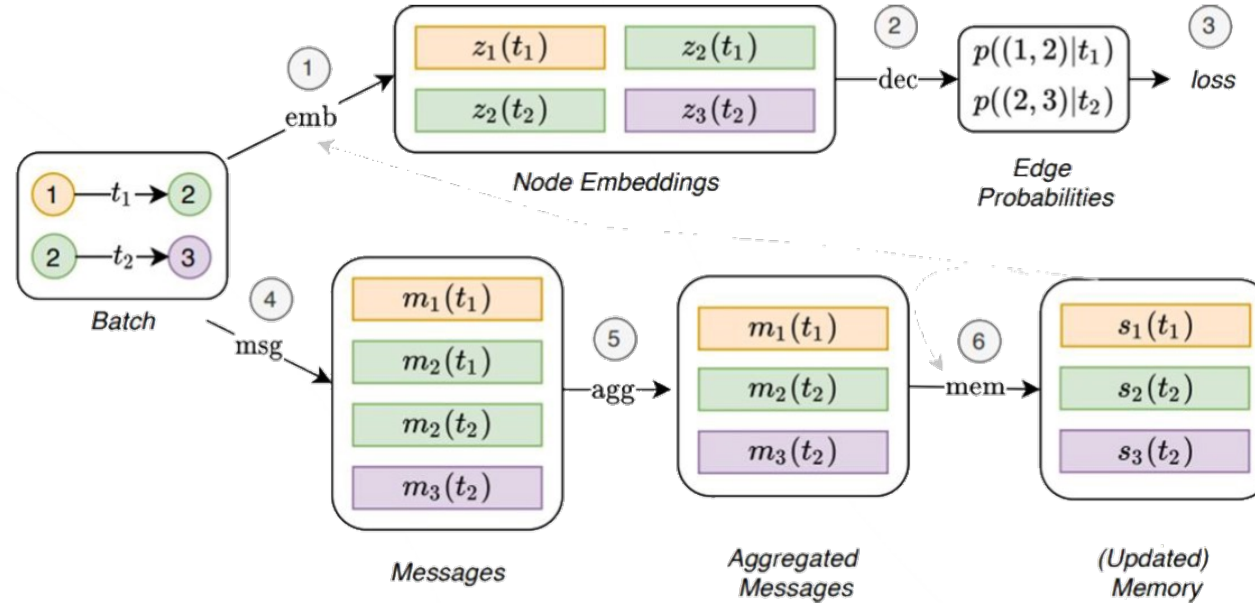
$$\mathbf{s}_i(t) = \text{mem}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-)).$$

$$\mathbf{z}_i(t) = \text{emb}(i, t) = \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t)),$$

TTG Model:

$$\begin{aligned} \mathbf{H}_i^{\text{node}} &= \text{RNN}^{\text{node}}(\mathbf{X}_{i, \leq T}), \quad \forall i \in \mathcal{V}, \\ \mathbf{H}_{i,j}^{\text{edge}} &= \text{RNN}^{\text{edge}}(\mathbf{A}_{i,j, \leq T}), \quad \forall (i, j) \in \mathcal{E}, \\ \mathbf{Z} &= \text{GNN}^L(\mathbf{H}^{\text{node}}, \mathbf{H}^{\text{edge}}). \end{aligned}$$

Architecture limitations: TGN



$$\mathbf{m}_i(t) = \text{msg}_s(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_d(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t))$$

$$\mathbf{m}_i(t) = \text{msg}_n(\mathbf{s}_i(t^-), t, \mathbf{v}_i(t)).$$

$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)).$$

$$\mathbf{s}_i(t) = \text{mem}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-)).$$

$$\mathbf{z}_i(t) = \text{emb}(i, t) \leftarrow \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t)),$$

TTG Model:

$$\mathbf{H}_i^{\text{node}} = \text{RNN}^{\text{node}}(\mathbf{X}_{i, \leq T}), \quad \forall i \in \mathcal{V},$$

$$\mathbf{H}_{i,j}^{\text{edge}} = \text{RNN}^{\text{edge}}(\mathbf{A}_{i,j, \leq T}), \quad \forall (i, j) \in \mathcal{E},$$

$$\mathbf{Z} = \text{GNN}^L(\mathbf{H}^{\text{node}}, \mathbf{H}^{\text{edge}}).$$

DyRep - динамика графа как Hawkes-процесс

Первая модель, которая **формально моделирует интенсивность событий**:

$$\lambda_{u,v}(t) = \text{softplus} \left((f_u(t)^T f_v(t)) \right)$$

События порождаются точечным процессом:

$$p((u, v, t) \mid \mathcal{H}_{t^-}) = \lambda_{u,v}(t) \exp \left(- \int_{t_{\text{last}}}^t \lambda_{u,v}(s) ds \right)$$

Функции обновления эмбеддингов:

- **association events** (должны быть частыми)
- **communication events** (взаимодействия)

Теоретические вопросы DGNN

Ограничения выразительности snapshot- vs event-based DGNN

GNN — это расширение обычных GNN, и поэтому они унаследовали их **структурные ограничения**.

- **Ограничение 1: Локальность**

Большинство моделей обновляют узел, используя k -hop окрестность.
Даже с динамикой это не меняется.

- **Ограничение 2: Перестановочная инвариантность**

Агрегаторы (sum/mean/max/attention) не различают перестановки соседей:
структурная выразительность остаётся ограниченной.

- **Ограничение 3: 1-WL верхняя граница**

Почти все стандартные GNN (GCN, GIN, GraphSAGE, attention-GNN)
не превосходят 1-WL по структурной выразительности.

Ограничения выразительности snapshot- vs event-based DGNN

GNN — это расширение обычных GNN, и поэтому они унаследовали их **структурные ограничения**.

Что это означает для DGNN?

Ни snapshot-based, ни event-based DGNN не превосходят 1-WL структурно, если не используют дополнительные источники информации:

- positional encodings,
- random features / hashing,
- подграфные методы (CAW),
- высокоуровневые структуры.

Важно: DGNN увеличивают **темпоральную** выразительность, но **не** структурную (если основа — message passing).

Memory decay (затухание памяти)

DGNN с памятью (TGN, DyRep, JODIE) используют обновление вида:

$$m_v(t) = \text{Cell}(m_v(t^-), \text{msg}(t))$$

У этого механизма есть важные ограничения:

Факт 1: Экспоненциальное затухание информации

Градиент обновления памяти:

$$\frac{\partial m_v(t)}{\partial m_v(t_k)}$$

обычно экспоненциально убывает,
как и в RNN/LSTM.

Memory decay (затухание памяти)

DGNN с памятью (TGN, DyRep, JODIE) используют обновление вида:

$$m_v(t) = \text{Cell}(m_v(t^-), \text{msg}(t))$$

У этого механизма есть важные ограничения:

Факт 2: Ограниченная размерность памяти

Если память имеет размер d ,

она не может хранить информацию о длинной последовательности событий без потерь (информационный bottleneck):

$$\text{history with } N \gg d \Rightarrow \text{information loss unavoidable}$$

Memory decay (затухание памяти)

DGNN с памятью (TGN, DyRep, JODIE) используют обновление вида:

$$m_v(t) = \text{Cell}(m_v(t^-), \text{msg}(t))$$

У этого механизма есть важные ограничения:

Факт 3: Марковость первого порядка

Большинство DGNN:

$$m_v(t) = F(m_v(t^-), \text{event})$$

То есть нет прямого доступа к истории до t^- .

Это снижает способность к долгосрочным прогнозам.

Нестационарность динамических графов

Динамические графы редко являются стационарными.

В общем случае:

$$p_t(e) \neq p_{t+1}(e)$$